

OKÜ Existing Application Integration Spec

Experience, Ticketing, Influencer Profile, Membership, and Event Commerce Module

1. Objective

This specification defines how to implement the discussed functionality inside the **existing OKÜ application** in Replit. The goal is **not** to build a standalone event app. The goal is to add a **modular experience-commerce subsystem** into the broader OKÜ website/app so that:

- OKÜ Super Admin controls all event creation and governance
- influencers manage only their own public profile content
- guests can discover, register for, and purchase curated experiences
- dynamic ticketing, add-ons, membership access, countdown releases, newsletters, attendee visibility, and seat visibility are all configurable per event
- the module plugs into the existing OKÜ hospitality ecosystem, user system, design system, and admin workflows

2. Product Scope Within Existing OKÜ App

This module should sit inside the current application as a **bounded domain** called:

experiences

This domain should integrate with the existing app in five ways:

1. Shared authentication

2. existing OKÜ users should not need separate login

3. use the current auth/session layer

4. Shared navigation

5. public users can discover experiences from the main OKÜ nav

6. admin users can manage experiences from admin nav

7. Shared user model

8. guest users, influencer users, and admin users all derive from the current user table/account system

9. Shared CMS / branding

10. event pages should inherit existing OKÜ visual language, typography, component styling, and SEO patterns

11. Shared analytics / CRM direction

12. event behavior, ticketing, newsletter interest, and membership behavior should feed into the broader OKÜ customer intelligence layer

3. Recommended Replit Architecture

If this is being developed inside Replit, keep implementation modular and incremental.

3.1 Recommended app structure

If current stack is React + Node, structure around:

- client/ or frontend/
- server/ or backend/
- shared/
- migrations/
- workers/ or jobs/

3.2 Recommended backend module boundaries

Add the following service/modules inside the backend:

- auth
- users
- profiles
- experiences
- tickets
- orders
- addons
- memberships
- newsletter
- notifications
- checkin
- analytics
- admin
- featureFlags

3.3 Recommended frontend feature boundaries

Add route-level modules/components for:

- public experience discovery
 - public event detail page
 - checkout flow
 - user dashboard
 - influencer profile editor
 - admin experience manager
 - admin ticket manager
 - admin reporting
-

4. Core Business Rules

These are non-negotiable rules that should be encoded in both backend authorization and frontend UX.

4.1 Event ownership

- every event is created by OKÜ admin only
- influencers cannot create events
- influencers cannot publish or modify event commercial settings
- all events are platform-owned records

4.2 Influencer permissions

Influencers can only manage: - profile image - cover image - display name rules if allowed - short bio - long bio - social links - selected profile descriptors/tags if allowed

Influencers cannot manage: - event title - event pricing - ticket inventory - event timing - add-ons - attendee list visibility - seat visibility - newsletters - membership rules

4.3 Public event configuration is admin-controlled

For each event, admin can configure: - available seats display on/off - attendee list display on/off - attendee list mode - early registration settings - countdown timer visibility - add-on availability - dynamic ticketing rules - membership gating - newsletter waitlist capture

5. User Roles

Use strict RBAC.

5.1 Roles

- `super_admin`
- `ops_admin`
- `marketing_admin`
- `finance_admin` (optional, for settlements/reporting)
- `influencer`
- `member`
- `guest`

5.2 Permission matrix

super_admin

Full access.

ops_admin

- create/update/publish/cancel events
- manage ticket rules
- manage availability display
- manage attendee visibility
- manage add-ons
- manage check-in
- manage assignment of influencers

marketing_admin

- manage event page media/content
- manage countdown settings
- manage newsletter segments
- manage event promotional copy
- manage featured placement

influencer

- read assigned event summaries
- update own profile only

member

- access member pricing or early access where allowed

guest

- browse, register, buy, join waitlist, subscribe

6. Database Design

Use the existing DB and add new tables. Prefer PostgreSQL.

6.1 Core tables

users

Assumed existing. Key fields expected: - id - email - password_hash or auth provider fields - role - status - created_at - updated_at

influencer_profiles

- id
- user_id FK -> users.id
- slug unique
- display_name
- headline
- short_bio
- long_bio
- profile_image_url
- cover_image_url
- instagram_url
- tiktok_url
- youtube_url
- website_url
- location
- is_public boolean default true
- is_verified boolean default false
- approval_status enum(approved , pending , rejected)
- created_at
- updated_at

influencer_profile_drafts

Use if moderation is required. - id - influencer_profile_id - submitted_by_user_id - payload_json - status enum(draft , submitted , approved , rejected) - reviewed_by_user_id nullable - review_notes nullable - created_at - updated_at

experiences

Main event table. - id - slug unique - title - subtitle - short_description - long_description - category - city - country - venue_name - venue_address - geo_lat - geo_lng - start_at - end_at - timezone - status enum(draft , scheduled , published , sold_out , completed , cancelled , archived) - hero_image_url - gallery_json - capacity_total - capacity_reserved - capacity_sold -

available_seats_display_mode enum(hidden , exact , approximate) -
 attendee_list_display_mode enum(hidden , public , buyers_only , partial) -
 show_countdown boolean default false - public_release_at nullable - early_release_at nullable
 - countdown_label nullable - newsletter_capture_enabled boolean default false -
 waitlist_enabled boolean default true - membership_rule_mode
 enum(none , members_only , members_early_access , members_discount) - is_featured
 boolean default false - seo_title - seo_description - created_by_user_id -
 updated_by_user_id - created_at - updated_at

experience_influencers

Join table. - id - experience_id - influencer_profile_id - role_label
 enum(featured_host , guest_host , partner_personality , special_guest) - sort_order -
 is_publicly_visible boolean default true - created_at

ticket_types

- id
- experience_id
- name
- description
- tier_code
- price_amount
- currency
- capacity
- sold_count
- min_per_order
- max_per_order
- sale_starts_at
- sale_ends_at
- visibility_mode
enum(hidden , visible , members_only , newsletter_only , invite_only)
- requires_membership boolean default false
- early_access_only boolean default false
- display_order
- status enum(draft , active , inactive , sold_out)
- created_at
- updated_at

ticket_pricing_rules

For dynamic ticketing. - id - ticket_type_id - rule_type
 enum(time_window , inventory_threshold , manual_override) - condition_json -
 action_json - priority - is_active - created_at

Example: - if remaining inventory < 20 then increase price by 10% - if current time > T then set price = X

experience_addons

- id
- experience_id
- name
- description
- price_amount
- currency
- capacity
- sold_count
- requires_ticket_type_id nullable
- requires_membership boolean default false
- is_active
- display_order
- created_at
- updated_at

orders

- id
- user_id
- experience_id
- subtotal_amount
- discount_amount
- fees_amount
- tax_amount
- total_amount
- currency
- status enum(pending , paid , failed , refunded , partially_refunded , cancelled)
- payment_provider
- payment_reference
- created_at
- updated_at

order_items

- id
- order_id
- item_type enum(ticket , addon)
- ticket_type_id nullable
- experience_addon_id nullable
- name_snapshot
- unit_price_amount
- quantity
- line_total_amount
- metadata_json

tickets

One row per issued seat. - `id` - `order_item_id` - `experience_id` - `ticket_type_id` - `owner_user_id` - `attendee_name` - `attendee_email` - `qr_code` - `status` enum(`issued`, `checked_in`, `cancelled`, `refunded`, `voided`) - `checked_in_at` nullable - `checked_in_by_user_id` nullable - `is_publicly_visible` boolean default false - `created_at`

experience_waitlist

- `id`
- `experience_id`
- `email`
- `user_id` nullable
- `source` enum(`event_page`, `countdown`, `newsletter`, `sold_out_page`)
- `status` enum(`active`, `notified`, `converted`, `expired`, `removed`)
- `created_at`

newsletter_subscriptions

Could reuse existing newsletter tables if present. - `id` - `email` - `user_id` nullable - `segment_key` - `source` - `is_active` - `created_at`

memberships

If membership already exists, integrate rather than duplicate. If not, add: - `id` - `user_id` - `tier` enum(`explorer`, `insider`, `patron`, `founder`) - `status` enum(`active`, `paused`, `expired`, `cancelled`) - `starts_at` - `ends_at` - `benefits_json` - `created_at`

experience_checkins

- `id`
- `ticket_id`
- `experience_id`
- `checked_in_by_user_id`
- `method` enum(`qr`, `manual`, `admin_override`)
- `created_at`

experience_analytics_daily

Materialized rollup table. - `id` - `experience_id` - `date` - `page_views` - `checkout_starts` - `orders_paid` - `tickets_sold` - `addon_units_sold` - `gross_revenue` - `waitlist_signups` - `newsletter_signups` - `member_purchases` - `created_at`

7. Backend API Design

Assuming Express or Nest-style JSON API.

7.1 Public APIs

Experiences

- GET /api/experiences
- filters: category, city, date_from, date_to, featured, influencer_slug
- GET /api/experiences/:slug
- GET /api/experiences/:id/attendees/public
- GET /api/experiences/:id/availability
- POST /api/experiences/:id/waitlist
- POST /api/experiences/:id/newsletter-subscribe

Checkout

- POST /api/checkout/quote
- POST /api/checkout/create-order
- POST /api/checkout/confirm-payment

Profiles

- GET /api/influencers/:slug

7.2 Authenticated user APIs

- GET /api/me/orders
- GET /api/me/tickets
- GET /api/me/membership
- PATCH /api/me/influencer-profile
- POST /api/me/influencer-profile/avatar
- POST /api/me/influencer-profile/cover

7.3 Admin APIs

- POST /api/admin/experiences
- PATCH /api/admin/experiences/:id
- POST /api/admin/experiences/:id/publish
- POST /api/admin/experiences/:id/cancel
- POST /api/admin/experiences/:id/feature
- POST /api/admin/experiences/:id/influencers
- POST /api/admin/experiences/:id/ticket-types
- PATCH /api/admin/ticket-types/:id
- POST /api/admin/experiences/:id/addons
- PATCH /api/admin/addons/:id

- GET /api/admin/experiences/:id/orders
 - GET /api/admin/experiences/:id/attendees
 - POST /api/admin/experiences/:id/checkin
 - GET /api/admin/analytics/experiences
-

8. Critical Backend Behaviors

8.1 Availability calculation

Availability must not trust frontend state. Backend should compute:

```
available = capacity_total - reserved_pending - sold_confirmed
```

For exact seat display: - return exact number

For approximate display: - map exact counts into labels such as: - Many seats available - Limited seats left - Almost sold out

8.2 Attendee list visibility rules

Public attendee list endpoint must enforce event-level rules.

- hidden: return nothing
- public: return public attendee subset
- buyers_only: return subset only if authenticated user has purchased
- partial: return explicitly visible attendees only

Each ticket record should store `is_publicly_visible` or derive it from attendee preference.

8.3 Countdown logic

Countdown should come from backend event config. Priority: 1. if current time < early_release_at => show countdown to early access if relevant user 2. else if current time < public_release_at => show countdown to public release 3. else no countdown

8.4 Early access logic

When quoting checkout, backend must validate whether user is allowed to purchase ticket type. Possible conditions: - active membership - newsletter-qualified segment - manual invite code - published sale start time reached

8.5 Dynamic pricing logic

All pricing must be resolved server-side. Order quote flow: 1. fetch ticket type 2. load active pricing rules 3. evaluate by priority 4. compute current unit price 5. lock quote for short TTL if needed

8.6 Add-on dependency logic

Examples: - add-on only available with VIP ticket - add-on only available while inventory remains - add-on only for members - one add-on per ticket max

Validation must occur on checkout quote and order confirmation.

9. Frontend Integration Plan

9.1 Public routes

Add these to the existing app router.

- `/experiences`
- `/experiences/:slug`
- `/influencers/:slug`
- `/checkout/:experienceSlug`
- `/my/tickets`
- `/my/orders`

9.2 Admin routes

- `/admin/experiences`
- `/admin/experiences/new`
- `/admin/experiences/:id`
- `/admin/experiences/:id/tickets`
- `/admin/experiences/:id/attendees`
- `/admin/experiences/:id/checkin`
- `/admin/analytics/experiences`

9.3 Influencer route

- `/account/influencer-profile`
-

10. Key UI Components

Public components

- `ExperienceCard`
- `ExperienceFilters`
- `ExperienceHero`
- `CountdownBanner`
- `TicketTierSelector`

- AddOnSelector
- AvailabilityBadge
- AttendeePreview
- WaitlistForm
- NewsletterSignupInline
- InfluencerProfileCard

Checkout components

- OrderSummary
- DynamicPriceRow
- MembershipBenefitCallout
- AddOnUpsell
- PaymentForm

Admin components

- ExperienceForm
- TicketTypeManager
- DynamicPricingRuleEditor
- AddOnManager
- VisibilitySettingsPanel
- InfluencerAssignmentPanel
- AttendeeTable
- CheckinScanner
- AnalyticsKPIGrid

Influencer components

- ProfileEditorForm
- ProfileImageUploader
- SocialLinksEditor
- PublicPreviewPanel

11. Admin Experience Form Requirements

The admin experience editor needs to be excellent because OKÜ controls the business.

Sections

1. Basic Info
2. Venue and Timing
3. Capacity and Visibility
4. Ticket Types
5. Dynamic Pricing Rules
6. Add-Ons

7. Influencer Assignment
8. Membership & Early Access
9. Newsletter / Waitlist
10. SEO / Publishing

Capacity and Visibility section fields

- total capacity
- show available seats? yes/no
- seats mode: hidden / exact / approximate
- show attendee list? yes/no
- attendee list mode: hidden / public / buyers_only / partial
- allow attendee opt-in for visibility? yes/no

Membership / Early Access fields

- members only event? yes/no
 - early access start datetime
 - public release start datetime
 - membership discount percentage
 - newsletter pre-register enabled? yes/no
-

12. Checkout Flow

Use a server-validated flow.

Steps

1. user selects ticket(s)
2. user selects add-ons
3. frontend requests quote from backend
4. backend validates access and computes current pricing
5. frontend displays quote
6. payment initiated
7. payment webhook confirms payment
8. order marked paid
9. ticket rows issued
10. email / QR notification sent

Important

Never decrement inventory permanently before payment confirmation. Use either: - short reservation hold records, or - transactional inventory decrement on payment success

For high-demand release events, short reservation holds are better.

13. Inventory / Concurrency Controls

This is critical.

Minimum requirements

- database transaction when confirming orders
- prevent overselling under concurrent traffic
- use row locking or inventory counters in transaction
- if using PostgreSQL, use `SELECT ... FOR UPDATE` on ticket inventory rows

Optional enhancement

Create `inventory_holds` table: - `id` - `experience_id` - `ticket_type_id` - `user_id` - `quantity` - `expires_at`

This is useful for countdown releases and highly demanded drops.

14. Newsletter and Early Registration Integration

If OKÜ already has a newsletter provider, integrate by API instead of duplicating.

Behavior

When a user clicks: - `Notify me` - `Join early access` - `Join waitlist`

Store locally and optionally sync outward.

Segments to support

- experience category interest
 - city interest
 - influencer interest
 - membership interest
 - sold-out waitlist
 - countdown reminder subscribers
-

15. Membership Integration

Do not build membership as an isolated feature. Tie it directly into quote and access logic.

Membership benefits to support

- early purchase access

- lower ticket price
- exclusive add-ons
- members-only events
- members-only presale inventory

Quote engine behavior

For logged-in user: 1. load active membership 2. determine eligible ticket types 3. determine eligible add-ons 4. apply discount if configured 5. expose member-specific messaging in response

16. Attendee Display Logic

Because you want optional attendee display, implement it carefully.

Recommendation

At checkout or post-purchase, ask attendee: - `Display my name on attendee list?` - `Display only first name + last initial?`

Rendering modes

- hidden: nothing shown
- public: render visible attendees
- buyers_only: only authenticated purchasers can see list
- partial: only featured/approved attendees or opt-in attendees

Privacy note for development

Never expose full emails or full customer identity publicly.

17. Available Seats Display Logic

This must be event-configurable and handled server-side.

Modes

hidden

No seat count shown.

exact

Display: - `23 seats remaining`

approximate

Display thresholds: - >50% remaining => `Seats available` - 20%-50% => `Limited seats remaining`
- <20% => `Almost sold out` - 0 => `Sold out`

These labels should be centralized in config.

18. Notification Jobs / Background Tasks

Use workers/cron/background jobs where needed.

Jobs to add

- send countdown reminder emails
- send release-now emails
- send waitlist conversion alerts
- send purchase confirmation emails
- send upcoming event reminders
- send post-event follow-up emails
- expire inventory holds
- roll up daily analytics

If Replit deployment has constraints, keep jobs simple and idempotent. If needed, use a lightweight queue backed by DB rather than complex infra initially.

19. Analytics Requirements

Need both event-level and platform-level visibility.

Event metrics

- page views
- ticket conversion rate
- tickets sold by tier
- add-on attach rate
- member purchases
- waitlist signups
- newsletter signups
- revenue by day
- seat velocity over time

Platform metrics

- top categories
- top cities

- repeat customer rate
- membership conversion rate
- sellout rate
- average order value

Recommended tracking events

- experience_viewed
- countdown_notified
- ticket_selected
- quote_requested
- checkout_started
- order_paid
- addon_added
- waitlist_joined
- newsletter_joined
- member_discount_applied

20. Integration With Broader OKÜ App

This is the most important section.

20.1 Shared homepage strategy

The main OKÜ homepage should surface curated experiences via reusable blocks: - featured experiences - happening soon - members-only releases - hosted by selected influencers

20.2 Shared account strategy

The existing account area should gain: - My Tickets - My Orders - My Membership - My Saved Experiences (optional) - Influencer Profile (only for influencer role)

20.3 Shared CRM/data strategy

Experience behavior should enrich the broader OKÜ customer model. Examples: - user attended culinary event - user likes wellness category - user converts on countdown drops - user is likely membership candidate

20.4 Shared editorial/content strategy

Event detail pages should support: - SEO metadata - OG images - structured content blocks - influencer cards linking into broader OKÜ content ecosystem

20.5 Shared admin strategy

Use existing admin shell/layout. Add a dedicated experiences section rather than creating a detached admin application.

21. Suggested Development Phases

Phase 1: Foundation

- DB migrations
- role checks
- influencer profile editor
- experience CRUD in admin
- public experiences list/detail page

Phase 2: Ticketing

- ticket types
- checkout quote engine
- payment integration
- order + ticket issuance
- availability display

Phase 3: Enhancements

- add-ons
- countdown release
- early access
- waitlist
- newsletter subscribe
- attendee display modes

Phase 4: Membership

- membership gating
- discounts
- members-only inventory
- member messaging

Phase 5: Operations + Intelligence

- QR check-in
- admin attendee dashboard
- analytics rollups
- scheduled notifications

22. Recommended Folder Structure

```
client/  
  src/  
    pages/  
      experiences/  
      influencers/  
      checkout/  
      account/  
      admin/experiences/  
    components/  
      experiences/  
      checkout/  
      influencer/  
      admin/  
    hooks/  
    api/  
    utils/  
  
  server/  
    modules/  
      experiences/  
        experiences.controller.ts  
        experiences.service.ts  
        experiences.repository.ts  
        experiences.validation.ts  
      tickets/  
      addons/  
      orders/  
      memberships/  
      newsletter/  
      profiles/  
      analytics/  
      admin/  
    middleware/  
    db/  
    jobs/  
    utils/  
  
  shared/  
    types/  
    constants/  
    schemas/
```

23. Validation Rules

Use shared schemas, ideally Zod if already using TypeScript.

Validate

- slugs unique
 - price ≥ 0
 - capacity non-negative
 - add-on capacity $\leq$ reasonable bound
 - sale dates valid
 - public release after early release where applicable
 - attendee display mode must be valid enum
 - available seats mode must be valid enum
 - influencer can only update own profile record
-

24. Security Requirements

- all admin endpoints protected by role middleware
- influencer profile endpoints scoped to authenticated user only
- rate limit waitlist and newsletter forms
- sanitize rich text content
- validate uploaded images
- never trust client pricing
- all payment confirmations processed by verified webhook
- log admin changes to event settings

Add an `audit_logs` table if not already present.

25. What Must Be Reused From Existing App

Before building, developers should inspect and reuse existing: - auth/session layer - file upload service - UI library/components - email service - payment integration if any - analytics/tracking library - admin shell/layout - SEO helpers - design tokens

Do not duplicate these unless there is a hard limitation.

26. Definition of Done for MVP

MVP is complete when: - admin can create and publish an experience - influencer can update own profile only - public users can browse and purchase tickets - admin can configure attendee visibility and seat visibility - countdown releases work - early registration works - newsletter capture works - add-ons work - membership benefits are applied if user qualifies - QR tickets are issued - admin can view attendees and check in guests

27. Final Technical Framing

This module should be implemented as:

a first-class experience-commerce domain inside the existing OKÜ application, with platform-owned event orchestration, influencer-limited profile self-service, configurable ticketing and visibility logic, and shared integration with OKÜ authentication, membership, analytics, and hospitality growth systems.